CASE STUDY 2 – PROGRESSIVE ELECTRICITY BILLING

1. Problem Statement

Design a Python program that calculates the monthly electricity bills of households using progressive consumption slabs, a fixed charge, a high-consumption surcharge and a low-consumption rebate. The program compares the bills of consecutive months and identifies unusual increases.

2. Objectives

1. Store household names and monthly units in a list of tuples.
2. Calculate the energy charge using progressive slabs.
3. Add the fixed charge and the surcharge, and subtract the rebate.
4. Compare every month with the previous month and calculate the percentage change.
5. Identify unusual increases in the bill.
6. Display a formatted bill table and the highest bill for every household.

3. Assumptions

- Slab rates: first 100 units Rs. 3.0, next 100 units Rs. 4.5, next 100 units Rs. 6.5 and above 300 units Rs. 8.0 per unit.
- The fixed charge is Rs. 50 per month.
- A surcharge of 5% of the energy charge applies when the units are more than 300.
- A rebate of 2% of (energy charge + fixed charge) applies when the units are 100 or less.
- An increase of more than 40% compared with the previous month is an unusual increase.
- Four months of sample data are used for every household.

4. Algorithm

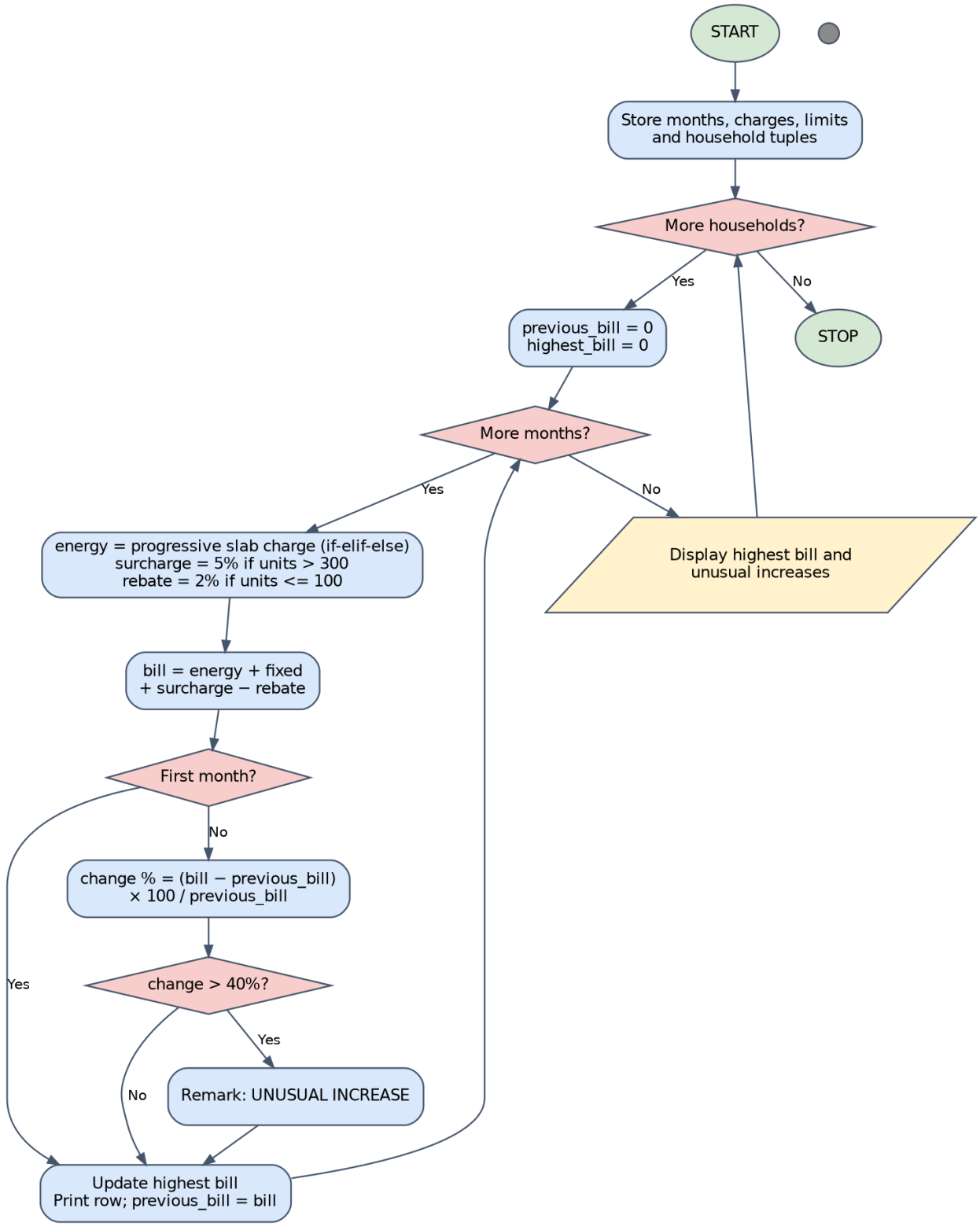
1. Store the months, charges, limits and the households as tuples (name, list of units).
2. For every household, set previous_bill = 0 and print the table heading.
3. For every month, calculate the energy charge using if-elif-else on the units.
4. Calculate the surcharge (units > 300) and the rebate (units <= 100).
5. bill = energy + fixed charge + surcharge − rebate.
6. From the second month, calculate the percentage change from the previous bill and mark it as UNUSUAL INCREASE when it is above 40%.
7. Track the highest bill and display the summary of the household.

5. Pseudocode

```
START
  SET months, fixed_charge = 50, surcharge_limit = 300, increase_limit = 40
  STORE households as (name, [units for each month])
  FOR each (household, units_list)
    previous_bill = 0, highest_bill = 0, unusual_count = 0
    FOR each month i
      units = units_list[i]
      IF units <= 100 THEN energy = units * 3.0
      ELSE IF units <= 200 THEN energy = 300 + (units - 100) * 4.5
      ELSE IF units <= 300 THEN energy = 750 + (units - 200) * 6.5
      ELSE energy = 1400 + (units - 300) * 8.0
      surcharge = 5% of energy IF units > surcharge_limit ELSE 0
      rebate = 2% of (energy + fixed_charge) IF units <= 100 ELSE 0
      bill = energy + fixed_charge + surcharge - rebate
      IF i > 0 THEN
        change = (bill - previous_bill) * 100 / previous_bill
        IF change > increase_limit THEN mark UNUSUAL INCREASE
      END IF
      update highest_bill
      PRINT row
      previous_bill = bill
    END FOR
    PRINT highest bill and unusual increases
  END FOR
```

```
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 2: Progressive Electricity Billing
months = ["Jan", "Feb", "Mar", "Apr"]
fixed_charge = 50
surcharge_limit = 300
increase_limit = 40

# (Household, [units consumed in each month])
households = [
    ("Anand", [90, 140, 180, 420]),
```

```
    ("Bhanu", [250, 260, 255, 270])
]

for household, units_list in households:
    print("HOUSEHOLD:", household)
    print("-" * 80)
    print(f"{'Month':<7}{'Units':>6}{'Energy':>10}{'Fixed':>8}{'Surcharge':>11}{'Rebate':>8}{'Bill':>10}{'Change':>9}

    previous_bill = 0
    highest_bill = 0
    highest_month = ""
    unusual_count = 0

    for i in range(len(months)):
        units = units_list[i]

        # Progressive slabs: 1-100 @3.0, 101-200 @4.5, 201-300 @6.5, above 300 @8.0
        if units <= 100:
            energy = units * 3.0
        elif units <= 200:
            energy = 100 * 3.0 + (units - 100) * 4.5
        elif units <= 300:
            energy = 100 * 3.0 + 100 * 4.5 + (units - 200) * 6.5
        else:
            energy = 100 * 3.0 + 100 * 4.5 + 100 * 6.5 + (units - 300) * 8.0

        surcharge = 0
        if units > surcharge_limit:
            surcharge = energy * 5 / 100

        rebate = 0
        if units <= 100:
            rebate = (energy + fixed_charge) * 2 / 100

        bill = energy + fixed_charge + surcharge - rebate

        remark = ""
        if i == 0:
            change_text = "-"
        else:
            change = (bill - previous_bill) * 100 / previous_bill
            change_text = f"{change:+.1f}%"
            if change > increase_limit:
                remark = "UNUSUAL INCREASE"
                unusual_count += 1

        if bill > highest_bill:
            highest_bill = bill
            highest_month = months[i]

        print(f"{months[i]:<7}{units:>6}{energy:>10.2f}{fixed_charge:>8.2f}{surcharge:>11.2f}{rebate:>8.2f}{bill:>10.2
        previous_bill = bill

    print(f"Highest bill: {highest_month} (Rs. {highest_bill:.2f}) | Unusual increases: {unusual_count}")
    print()
```

8. Test Cases

Test Case	Units / Scenario	Expected Result
1	90 units (Anand, Jan)	Energy Rs. 270.00, rebate Rs. 6.40, bill Rs. 313.60
2	100 units	Energy Rs. 300.00, rebate Rs. 7.00, bill Rs. 343.00
3	200 units	Energy Rs. 750.00 (end of second slab), bill Rs. 800.00
4	300 units	Energy Rs. 1400.00, no surcharge, bill Rs. 1450.00
5	301 units	Energy Rs. 1408.00, surcharge Rs. 70.40, bill Rs. 1528.40
6	Anand: Jan → Feb (313.60 → 530.00)	+69.0% - UNUSUAL INCREASE
7	Anand: Feb → Mar (530.00 → 710.00)	+34.0% - not flagged (below the 40% limit)

Test Case	Units / Scenario	Expected Result
8	Bhanu: all four months	No unusual increase (largest change +8.4%)

9. Sample Output

```
HOUSEHOLD: Anand
--------------------------------------------------------------------------------
Month   Units    Energy   Fixed  Surcharge  Rebate      Bill   Change  Remark
Jan        90    270.00   50.00       0.00    6.40    313.60        -
Feb       140    480.00   50.00       0.00    0.00    530.00   +69.0%  UNUSUAL INCREASE
Mar       180    660.00   50.00       0.00    0.00    710.00   +34.0%
Apr       420   2360.00   50.00     118.00    0.00   2528.00  +256.1%  UNUSUAL INCREASE
Highest bill: Apr (Rs. 2528.00) | Unusual increases: 2

HOUSEHOLD: Bhanu
--------------------------------------------------------------------------------
Month   Units    Energy   Fixed  Surcharge  Rebate      Bill   Change  Remark
Jan       250   1075.00   50.00       0.00    0.00   1125.00        -
Feb       260   1140.00   50.00       0.00    0.00   1190.00    +5.8%
Mar       255   1107.50   50.00       0.00    0.00   1157.50    -2.7%
Apr       270   1205.00   50.00       0.00    0.00   1255.00    +8.4%
Highest bill: Apr (Rs. 1255.00) | Unusual increases: 0
```

10. Conclusion

The Progressive Electricity Billing program uses lists, tuples, tuple unpacking, if-elif-else ladders and loops to calculate slab-wise bills. Comparing every month with the previous bill shows how loops and conditions can detect unusual consumption patterns.

CASE STUDY 4 – EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS

1. Problem Statement

Design a Python program that calculates the gross salary of employees from basic pay, allowances, overtime and incentives, applies progressive deductions and generates a formatted salary statement for every employee along with a payroll summary.

2. Objectives

1. Store employee details in a list of tuples.
2. Calculate overtime pay and gross salary.
3. Calculate provident fund, professional tax and progressive income tax.
4. Calculate total deductions and net salary.
5. Print a formatted salary statement for every employee.
6. Find the total payroll and the highest net salary.

3. Assumptions

- Overtime is paid at Rs. 150 per hour.
- Provident fund is 12% of the basic pay.
- Professional tax: Rs. 0 up to a gross of 15,000, Rs. 150 up to 25,000 and Rs. 200 above 25,000.
- Progressive income tax on the gross salary: 0% up to 25,000, 5% on the next 25,000 and 10% on the amount above 50,000.
- Employee data is sample data stored in the program.

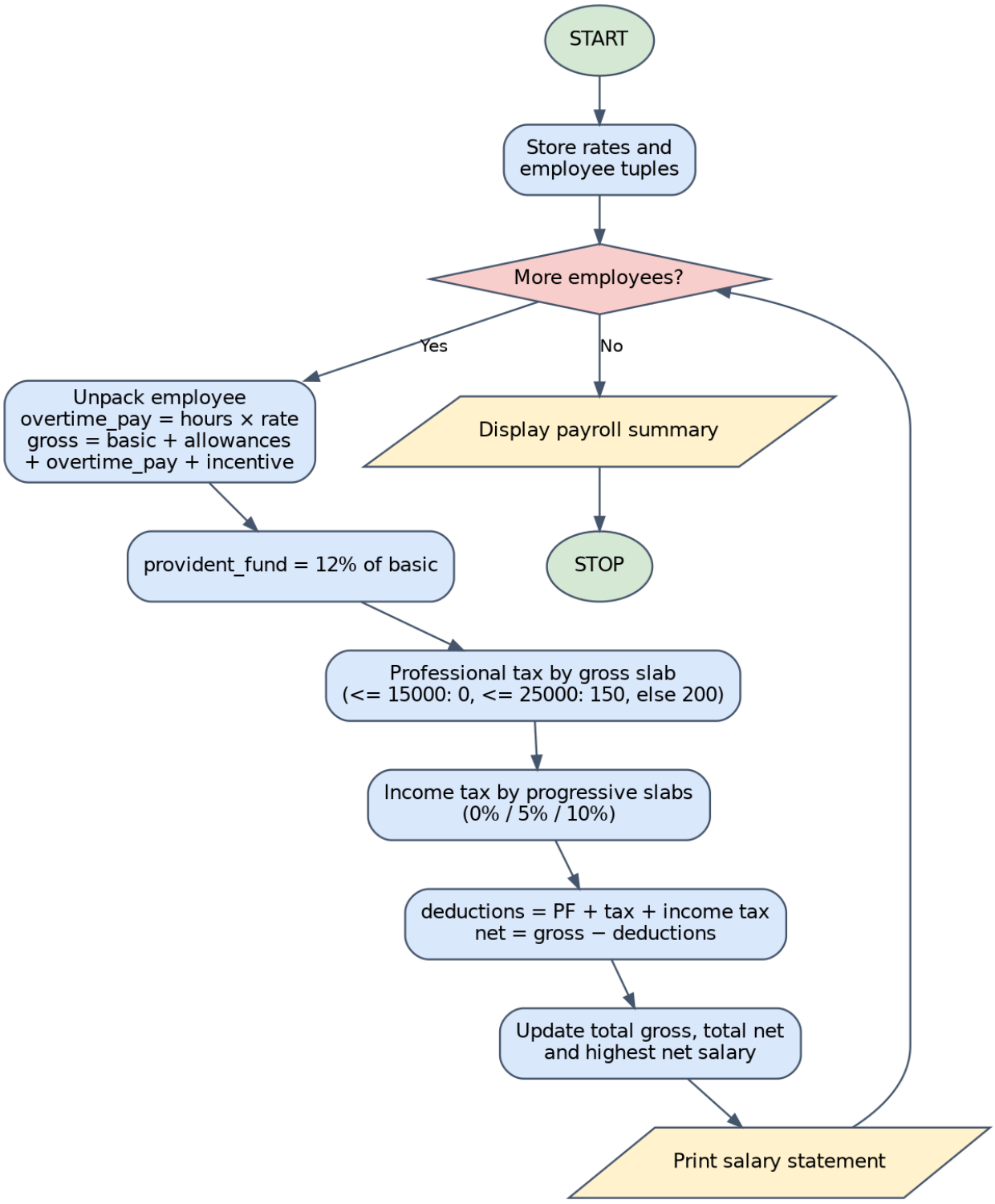
4. Algorithm

1. Store the employees as tuples and set the overtime rate and PF percentage.
2. Unpack each employee and calculate overtime pay and gross salary.
3. Calculate the provident fund.
4. Select the professional tax using if-elif-else on the gross salary.
5. Calculate the progressive income tax using slab conditions.
6. Calculate total deductions and net salary.
7. Update the totals and the highest net salary.
8. Print the salary statement and, after the loop, the payroll summary.

5. Pseudocode

```
START
  SET overtime_rate = 150, pf_percent = 12
  STORE employees as (name, basic, allowances, overtime_hours, incentive)
  total_gross = 0, total_net = 0, top_net = 0
  FOR each employee
    overtime_pay = overtime_hours * overtime_rate
    gross = basic + allowances + overtime_pay + incentive
    provident_fund = basic * pf_percent / 100
    IF gross <= 15000 THEN professional_tax = 0
    ELSE IF gross <= 25000 THEN professional_tax = 150
    ELSE professional_tax = 200
    IF gross <= 25000 THEN income_tax = 0
    ELSE IF gross <= 50000 THEN income_tax = (gross - 25000) * 5 / 100
    ELSE income_tax = 25000 * 5 / 100 + (gross - 50000) * 10 / 100
    deductions = provident_fund + professional_tax + income_tax
    net = gross - deductions
    update total_gross, total_net and highest net salary
    PRINT salary statement
  END FOR
  PRINT payroll summary
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 4: Employee Payroll and Deduction Analysis
overtime_rate = 150
pf_percent = 12

# (Name, Basic Pay, Allowances, Overtime Hours, Incentive)
employees = [
    ("Ravi", 20000, 5000, 10, 2000),
    ("Sita", 35000, 8000, 5, 5000),
    ("Kiran", 60000, 12000, 0, 10000),
    ("Meena", 12000, 2000, 20, 0)
]

total_gross = 0
```

```
total_net = 0
top_name = ""
top_net = 0

for name, basic, allowances, overtime_hours, incentive in employees:
    overtime_pay = overtime_hours * overtime_rate
    gross = basic + allowances + overtime_pay + incentive

    provident_fund = basic * pf_percent / 100

    if gross <= 15000:
        professional_tax = 0
    elif gross <= 25000:
        professional_tax = 150
    else:
        professional_tax = 200

    # Progressive income tax: 0% up to 25000, 5% on the next 25000, 10% above 50000
    if gross <= 25000:
        income_tax = 0
    elif gross <= 50000:
        income_tax = (gross - 25000) * 5 / 100
    else:
        income_tax = 25000 * 5 / 100 + (gross - 50000) * 10 / 100

    deductions = provident_fund + professional_tax + income_tax
    net = gross - deductions

    total_gross += gross
    total_net += net
    if net > top_net:
        top_net = net
        top_name = name

    print("SALARY STATEMENT -", name)
    print("-" * 44)
    print(f"{'Basic Pay':<32}{basic:>12.2f}")
    print(f"{'Allowances':<32}{allowances:>12.2f}")
    print(f"{'Overtime (' + str(overtime_hours) + ' hours)':<32}{overtime_pay:>12.2f}")
    print(f"{'Incentive':<32}{incentive:>12.2f}")
    print(f"{'GROSS SALARY':<32}{gross:>12.2f}")
    print(f"{'Provident Fund (12% of basic)':<32}{provident_fund:>12.2f}")
    print(f"{'Professional Tax':<32}{professional_tax:>12.2f}")
    print(f"{'Income Tax (progressive)':<32}{income_tax:>12.2f}")
    print(f"{'TOTAL DEDUCTIONS':<32}{deductions:>12.2f}")
    print(f"{'NET SALARY':<32}{net:>12.2f}")
    print()

print("PAYROLL SUMMARY")
print("-" * 44)
print(f"{'Total gross salary':<32}{total_gross:>12.2f}")
print(f"{'Total net salary':<32}{total_net:>12.2f}")
print(f"Highest net salary: {top_name} (Rs. {top_net:.2f})")
```

8. Test Cases

Test Case	Employee / Scenario	Expected Result
1	Ravi - gross Rs. 28500	PF 2400, professional tax 200, income tax 175 → net Rs. 25725.00
2	Sita - gross Rs. 48750	Income tax 1187.50 → net Rs. 43162.50
3	Kiran - gross Rs. 82000	Income tax 4450.00 (1250 + 10% of 32000) → net Rs. 70150.00
4	Meena - gross Rs. 17000	Professional tax 150, no income tax → net Rs. 15410.00
5	Gross Rs. 10000 (up to 15000)	Professional tax 0, income tax 0 → net Rs. 9040.00
6	Gross exactly Rs. 50000	Income tax 1250.00 → net Rs. 42550.00
7	Payroll summary	Total gross 176250.00, total net 154447.50, highest net salary: Kiran

9. Sample Output

```
SALARY STATEMENT - Ravi
-----------------------------------------
Basic Pay                           20000.00
Allowances                           5000.00
Overtime (10 hours)                  1500.00
Incentive                            2000.00
GROSS SALARY                        28500.00
Provident Fund (12% of basic)        2400.00
Professional Tax                      200.00
Income Tax (progressive)              175.00
TOTAL DEDUCTIONS                     2775.00
NET SALARY                          25725.00

SALARY STATEMENT - Sita
-----------------------------------------
Basic Pay                           35000.00
Allowances                           8000.00
Overtime (5 hours)                    750.00
Incentive                            5000.00
GROSS SALARY                        48750.00
Provident Fund (12% of basic)        4200.00
Professional Tax                      200.00
Income Tax (progressive)             1187.50
TOTAL DEDUCTIONS                     5587.50
NET SALARY                          43162.50

SALARY STATEMENT - Kiran
-----------------------------------------
Basic Pay                           60000.00
Allowances                          12000.00
Overtime (0 hours)                      0.00
Incentive                           10000.00
GROSS SALARY                        82000.00
Provident Fund (12% of basic)        7200.00
Professional Tax                      200.00
Income Tax (progressive)             4450.00
TOTAL DEDUCTIONS                    11850.00
NET SALARY                          70150.00

SALARY STATEMENT - Meena
-----------------------------------------
Basic Pay                           12000.00
Allowances                           2000.00
Overtime (20 hours)                  3000.00
Incentive                               0.00
GROSS SALARY                        17000.00
Provident Fund (12% of basic)        1440.00
Professional Tax                      150.00
Income Tax (progressive)                0.00
TOTAL DEDUCTIONS                     1590.00
NET SALARY                          15410.00

PAYROLL SUMMARY
-----------------------------------------
Total gross salary                 176250.00
Total net salary                   154447.50
Highest net salary: Kiran (Rs. 70150.00)
```

10. Conclusion

The Payroll program uses a list of tuples, tuple unpacking, arithmetic operators and if-elif-else ladders to calculate gross salary, progressive deductions and net salary. Formatted strings are used to print a neat salary statement for every employee.

CASE STUDY 11 – EXAMINATION RESULT AND GRACE-MARK VALIDATION

1. Problem Statement

Design a Python program that processes the marks of students in several subjects, validates the marks, applies a grace-mark policy to eligible students without exceeding the maximum marks, and generates the final result and classification.

2. Objectives

1. Store student names and subject marks using tuples and lists.
2. Validate that every mark is between 0 and the maximum marks.
3. Apply subject-wise pass criteria.
4. Apply the grace-mark policy to eligible students only.
5. Generate the final classification of every student.

3. Assumptions

- There are five subjects and the maximum mark in each subject is 100; the pass mark in each subject is 40.
- Grace marks can be given only when the student has failed in at most 2 subjects and the total marks needed to pass those subjects is 5 or less.
- Grace marks are used only to raise failed subjects up to 40 marks; no mark can exceed 100.
- Classification of passed students: 75% or more Distinction, 60% to 74.99% First Class, 50% to 59.99% Second Class and below 50% Pass Class.
- Marks outside the range 0-100 make the record invalid.

4. Algorithm

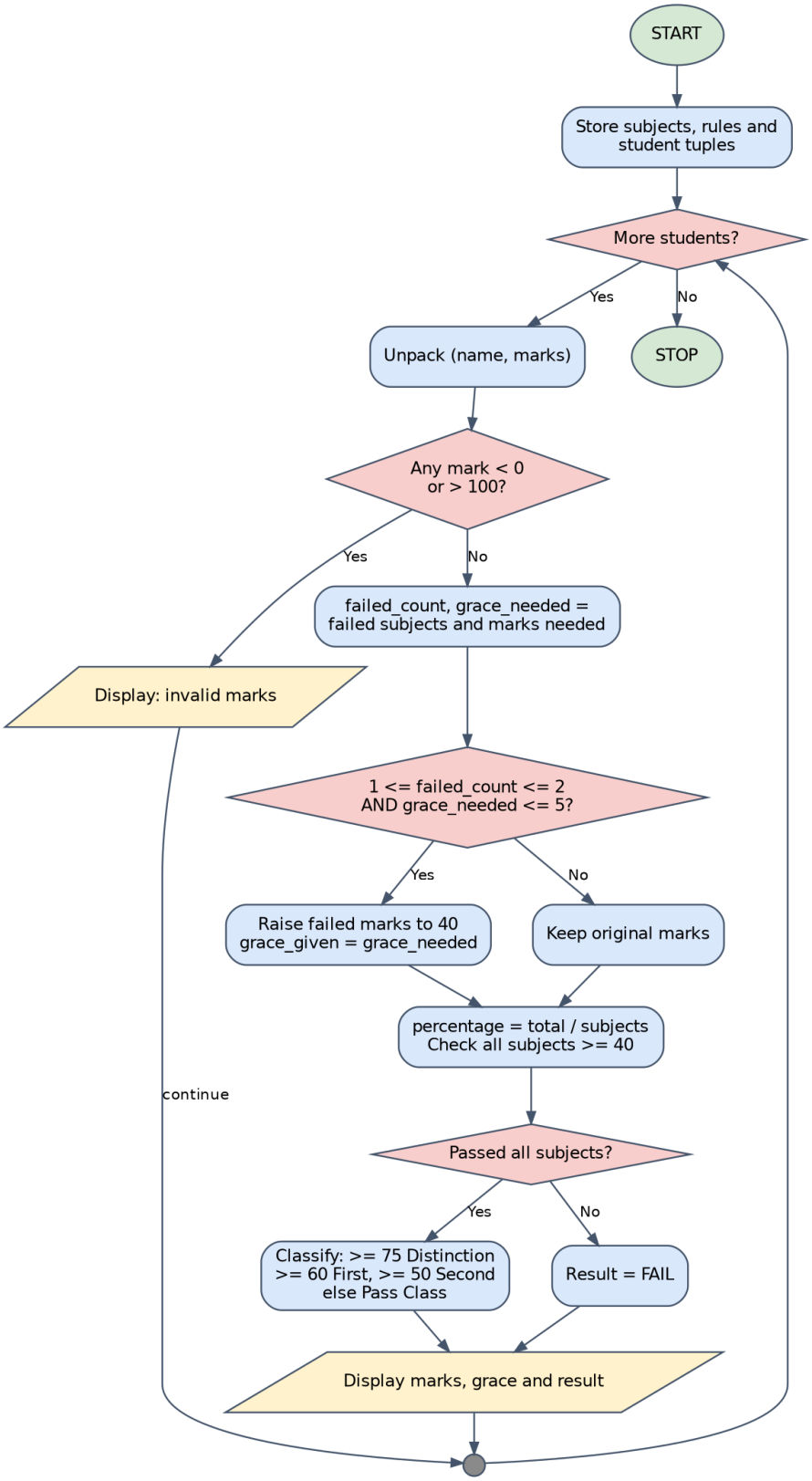
1. Store the subjects, rules and the student tuples (name, marks list).
2. Check every mark of the student; if any mark is below 0 or above 100, report invalid marks and continue with the next student.
3. Count the failed subjects and the total grace marks needed to reach the pass mark.
4. If the failed subjects are 1 or 2 and the grace needed is at most 5, raise the failed marks to 40 and record the grace used; otherwise keep the marks unchanged.
5. Calculate the total and percentage of the final marks and check that all subjects are passed.
6. Classify the student as Distinction, First Class, Second Class, Pass Class or FAIL.
7. Display the original marks, final marks, grace used and the result.

5. Pseudocode

```
START
  SET pass_mark = 40, max_marks = 100, max_grace = 5
  STORE students as (name, marks)
  FOR each (name, marks)
    valid = TRUE
    FOR each mark in marks
      IF mark < 0 OR mark > max_marks THEN valid = FALSE
    END FOR
    IF NOT valid THEN
      PRINT "Invalid marks"
      CONTINUE
    END IF
    failed_count = 0, grace_needed = 0
    FOR each mark in marks
      IF mark < pass_mark THEN
        failed_count = failed_count + 1
        grace_needed = grace_needed + (pass_mark - mark)
      END IF
    END FOR
    IF failed_count >= 1 AND failed_count <= 2 AND grace_needed <= max_grace THEN
      raise every failed mark to pass_mark (maximum limit max_marks)
```

```
      grace_given = grace_needed
    ELSE
      final_marks = marks
    END IF
    passed = all final marks >= pass_mark
    percentage = total of final marks / number of subjects
    IF NOT passed THEN result = "FAIL"
    ELSE classify using 75, 60 and 50
    PRINT original marks, final marks, grace and result
  END FOR
STOP
```

6. Flowchart


START
Store subjects, rules and student tuples
More students?
Yes
No
Unpack (name, marks)
STOP
Any mark < 0 or > 100?
Yes
No
failed_count, grace_needed = failed subjects and marks needed
Display: invalid marks
1 <= failed_count <= 2 AND grace_needed <= 5?
Yes
No
Raise failed marks to 40 grace_given = grace_needed
Keep original marks
percentage = total / subjects Check all subjects >= 40
continue
Passed all subjects?
Yes
No
Classify: >= 75 Distinction >= 60 First, >= 50 Second else Pass Class
Result = FAIL
Display marks, grace and result

7. Python Source Code

```
# Case Study 11: Examination Result and Grace-Mark Validation
subjects = ["Python", "Maths", "Web", "English", "GenAI"]
max_marks = 100
pass_mark = 40
max_grace = 5

# (Student, [marks in every subject])
students = [
    ("Anu", [78, 82, 91, 70, 85]),
    ("Bala", [38, 55, 62, 40, 58]),
    ("Chandu", [36, 37, 70, 65, 60]),
    ("Divya", [39, 39, 50, 52, 45]),
    ("Esha", [20, 80, 85, 90, 75]),
    ("Farhan", [45, 105, 60, 70, 55])
]

print("EXAMINATION RESULTS")
print("-" * 72)

for name, marks in students:
    # Validation of marks
    valid = True
    for mark in marks:
        if mark < 0 or mark > max_marks:
            valid = False

    if not valid:
        print(f"{name:<8} INVALID MARKS ENTERED (marks must be between 0 and {max_marks})")
        print()
        continue

    # Grace-mark policy: at most 2 failed subjects and at most 5 marks needed in total
    failed_count = 0
    grace_needed = 0
    for mark in marks:
        if mark < pass_mark:
            failed_count += 1
            grace_needed += pass_mark - mark

    grace_given = 0
    final_marks = []
    if failed_count > 0 and failed_count <= 2 and grace_needed <= max_grace:
        grace_given = grace_needed
        for mark in marks:
            if mark < pass_mark:
                mark = pass_mark
            if mark > max_marks:
                mark = max_marks
            final_marks.append(mark)
    else:
        final_marks = marks

    # Final result
    passed = True
    total = 0
    for mark in final_marks:
        total += mark
        if mark < pass_mark:
            passed = False
    percentage = total / len(subjects)

    if not passed:
        classification = "FAIL"
    elif percentage >= 75:
        classification = "PASS - Distinction"
    elif percentage >= 60:
        classification = "PASS - First Class"
    elif percentage >= 50:
        classification = "PASS - Second Class"
    else:
        classification = "PASS - Pass Class"

    print(f"{name:<8} Original: {marks}")
    print(f"{'':<8} Final   : {final_marks}")
    print(f"{'':<8} Grace marks used: {grace_given} | Percentage: {percentage:.2f}%")
    print(f"{'':<8} Result  : {classification}")
```

```
    print()
```

8. Test Cases

Test Case	Student / Marks	Expected Result
1	Anu - 78, 82, 91, 70, 85	No grace needed; 81.20% - PASS - Distinction
2	Bala - 38, 55, 62, 40, 58	1 failed subject needs 2 marks → grace 2; Python becomes 40; 51.00% - PASS - Second Class
3	Chandu - 36, 37, 70, 65, 60	2 failed subjects need 4 + 3 = 7 marks (more than 5) → no grace; FAIL
4	Divya - 39, 39, 50, 52, 45	2 failed subjects need 1 + 1 = 2 marks → grace 2; 45.40% - PASS - Pass Class
5	Esha - 20, 80, 85, 90, 75	1 failed subject needs 20 marks → no grace; FAIL
6	Farhan - 45, 105, 60, 70, 55	INVALID MARKS ENTERED (105 is above 100)

9. Sample Output

```
EXAMINATION RESULTS
------------------------------------------------------------------------
Anu      Original: [78, 82, 91, 70, 85]
         Final   : [78, 82, 91, 70, 85]
         Grace marks used: 0 | Percentage: 81.20%
         Result  : PASS - Distinction

Bala     Original: [38, 55, 62, 40, 58]
         Final   : [40, 55, 62, 40, 58]
         Grace marks used: 2 | Percentage: 51.00%
         Result  : PASS - Second Class

Chandu   Original: [36, 37, 70, 65, 60]
         Final   : [36, 37, 70, 65, 60]
         Grace marks used: 0 | Percentage: 53.60%
         Result  : FAIL

Divya    Original: [39, 39, 50, 52, 45]
         Final   : [40, 40, 50, 52, 45]
         Grace marks used: 2 | Percentage: 45.40%
         Result  : PASS - Pass Class

Esha     Original: [20, 80, 85, 90, 75]
         Final   : [20, 80, 85, 90, 75]
         Grace marks used: 0 | Percentage: 70.00%
         Result  : FAIL

Farhan   INVALID MARKS ENTERED (marks must be between 0 and 100)
```

10. Conclusion

The Examination Result program combines tuples, lists, nested loops, continue and logical operators to validate marks, apply a controlled grace-mark policy and classify the final result. The checks prevent invalid data and make sure that no mark exceeds the maximum.

CASE STUDY 16 – MULTI-DAY WATER TANK SIMULATION

1. Problem Statement

Design a Python program that simulates a water tank over several days using changing inflow and outflow quantities. The program updates the water level, detects overflow and shortage, and finds the first day on which the tank reaches a specified target level.

2. Objectives

1. Store daily inflow and outflow quantities in lists.
2. Update the water level every day.
3. Detect overflow when the level exceeds the tank capacity.
4. Detect shortage when the outflow is more than the available water.
5. Find the first day on which the target level is reached.
6. Display a daily table and a summary.

3. Assumptions

- Tank capacity is 10,000 litres, the opening level is 4,000 litres and the target level is 9,000 litres.
- Each day, the inflow is added first and the outflow is removed afterwards.
- Water above the capacity is lost as overflow; water that cannot be supplied is counted as shortage and the level never goes below zero.
- The target is reached when the closing level of a day is 9,000 litres or more.
- A closing level below 1,000 litres is shown as a low level.

4. Algorithm

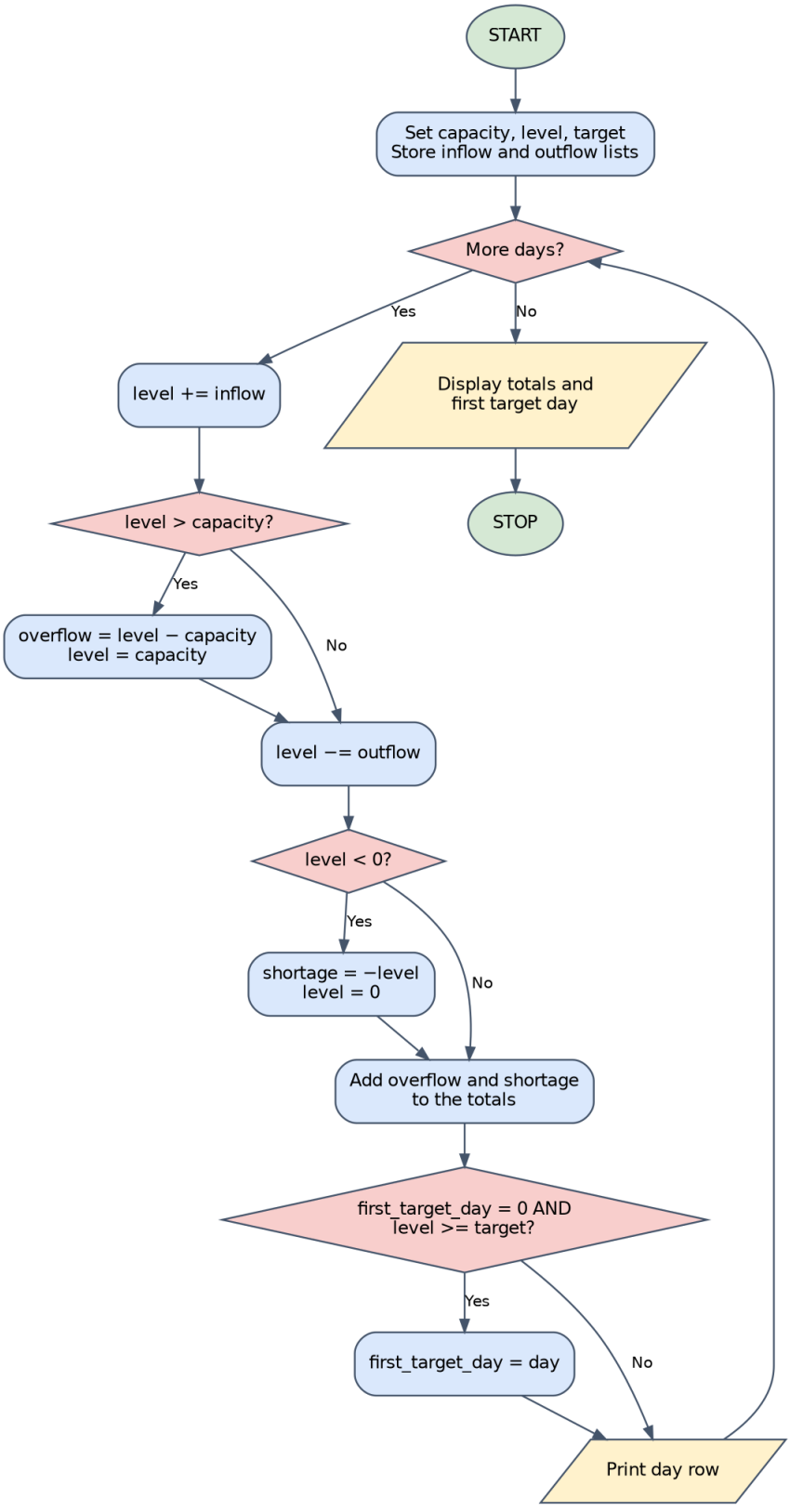
1. Store the capacity, opening level, target level and the inflow and outflow lists.
2. For every day, add the inflow to the level.
3. If the level is above the capacity, store the extra water as overflow and set the level to the capacity.
4. Subtract the outflow; if the level becomes negative, store the missing water as shortage and set the level to zero.
5. Add the overflow and shortage to the totals.
6. If the target day is not yet found and the closing level is at least the target, store the day number.
7. Print the day row with a remark; after the loop display the totals and the first target day.

5. Pseudocode

```
START
  SET capacity = 10000, level = 4000, target_level = 9000, low_level = 1000
  STORE inflow list and outflow list
  total_overflow = 0, total_shortage = 0, first_target_day = 0
  FOR each day i from 0 to number of days - 1
    level = level + inflow[i]
    overflow = 0
    IF level > capacity THEN
      overflow = level - capacity
      level = capacity
    END IF
    level = level - outflow[i]
    shortage = 0
    IF level < 0 THEN
      shortage = -level
      level = 0
    END IF
    total_overflow = total_overflow + overflow
    total_shortage = total_shortage + shortage
    IF first_target_day = 0 AND level >= target_level THEN
      first_target_day = i + 1
      remark = "TARGET REACHED"
    ELSE remark = Overflow / SHORTAGE / Low level when applicable
    PRINT day row
  END FOR
  PRINT total_overflow, total_shortage, first_target_day
```

```
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 16: Multi-Day Water Tank Simulation
capacity = 10000
level = 4000
target_level = 9000
low_level = 1000

# Litres entering and leaving the tank on each day
inflow = [3000, 4000, 6000, 2000, 5000, 1500, 3000]
outflow = [2500, 3000, 1000, 4500, 3000, 9000, 6000]

total_overflow = 0
total_shortage = 0
first_target_day = 0

print(f"Tank capacity: {capacity} L | Opening level: {level} L | Target level: {target_level} L")
print("-" * 76)
print(f"{'Day':<5}{'Inflow':>8}{'Outflow':>9}{'Overflow':>10}{'Shortage':>10}{'Closing':>9}   Remark")

for i in range(len(inflow)):
    day = i + 1

    level += inflow[i]
    overflow = 0
    if level > capacity:
        overflow = level - capacity
        level = capacity

    level -= outflow[i]
    shortage = 0
    if level < 0:
        shortage = -level
        level = 0

    total_overflow += overflow
    total_shortage += shortage

    remark = ""
    if first_target_day == 0 and level >= target_level:
        first_target_day = day
        remark = "TARGET REACHED"
    elif overflow > 0:
        remark = "Overflow"
    elif shortage > 0:
        remark = "SHORTAGE"
    elif level < low_level:
        remark = "Low level"

    print(f"{day:<5}{inflow[i]:>8}{outflow[i]:>9}{overflow:>10}{shortage:>10}{level:>9}   {remark}")

print("-" * 76)
print("Total overflow :", total_overflow, "L")
print("Total shortage :", total_shortage, "L")
if first_target_day > 0:
    print("Target level first reached on day", first_target_day)
else:
    print("Target level was never reached")
```

8. Test Cases

Test Case	Day / Scenario	Expected Result
1	Day 1 (inflow 3000, outflow 2500)	Closing level 4500 L, no remark
2	Day 3 (inflow 6000, outflow 1000)	Overflow 1500 L; closing level 9000 L - TARGET REACHED
3	Day 4 (inflow 2000, outflow 4500)	Overflow 1000 L; closing level 5500 L
4	Day 6 (inflow 1500, outflow 9000)	Shortage 500 L; closing level 0 L - SHORTAGE
5	Day 7 (inflow 3000, outflow 6000)	Shortage 3000 L; closing level 0 L
6	Totals	Overflow 3000 L, shortage 3500 L, target first reached on day 3

9. Sample Output

```
Tank capacity: 10000 L | Opening level: 4000 L | Target level: 9000 L
-------------------------------------------------------------------------------
Day    Inflow  Outflow  Overflow  Shortage  Closing   Remark
1        3000     2500         0         0     4500
2        4000     3000         0         0     5500
3        6000     1000      1500         0     9000   TARGET REACHED
4        2000     4500      1000         0     5500   Overflow
5        5000     3000       500         0     7000   Overflow
6        1500     9000         0       500        0   SHORTAGE
7        3000     6000         0      3000        0   SHORTAGE
-------------------------------------------------------------------------------
Total overflow : 3000 L
Total shortage : 3500 L
Target level first reached on day 3
```

10. Conclusion

The Water Tank Simulation program uses lists, indexing, loops and conditional statements to update the water level day by day. The flag variable first_target_day shows how a program can remember the first occurrence of an event while the loop continues.

CASE STUDY 31 – STRING-BASED EXAMINATION ANSWER EVALUATION

1. Problem Statement

Design a Python program that compares the answer strings of students with an answer key, counts correct, wrong and unanswered responses, applies negative marking and produces a student-wise result.

2. Objectives

1. Store the answer key and the student answers as strings.
2. Compare every answer with the key using string indexing.
3. Count correct, incorrect and unanswered responses.
4. Apply negative marking and calculate the score.
5. Reject answer strings of the wrong length.
6. Display a student-wise result and the highest score.

3. Assumptions

- The answer key is ABCDABCDAB (10 questions).
- A correct answer gives 4 marks, a wrong answer deducts 1 mark and an unanswered question (shown as a dash '-') gives 0 marks.
- The pass score is 16 out of 40.
- Answers may be typed in lower case; they are converted to upper case before checking.
- An answer string whose length is different from the key is invalid.

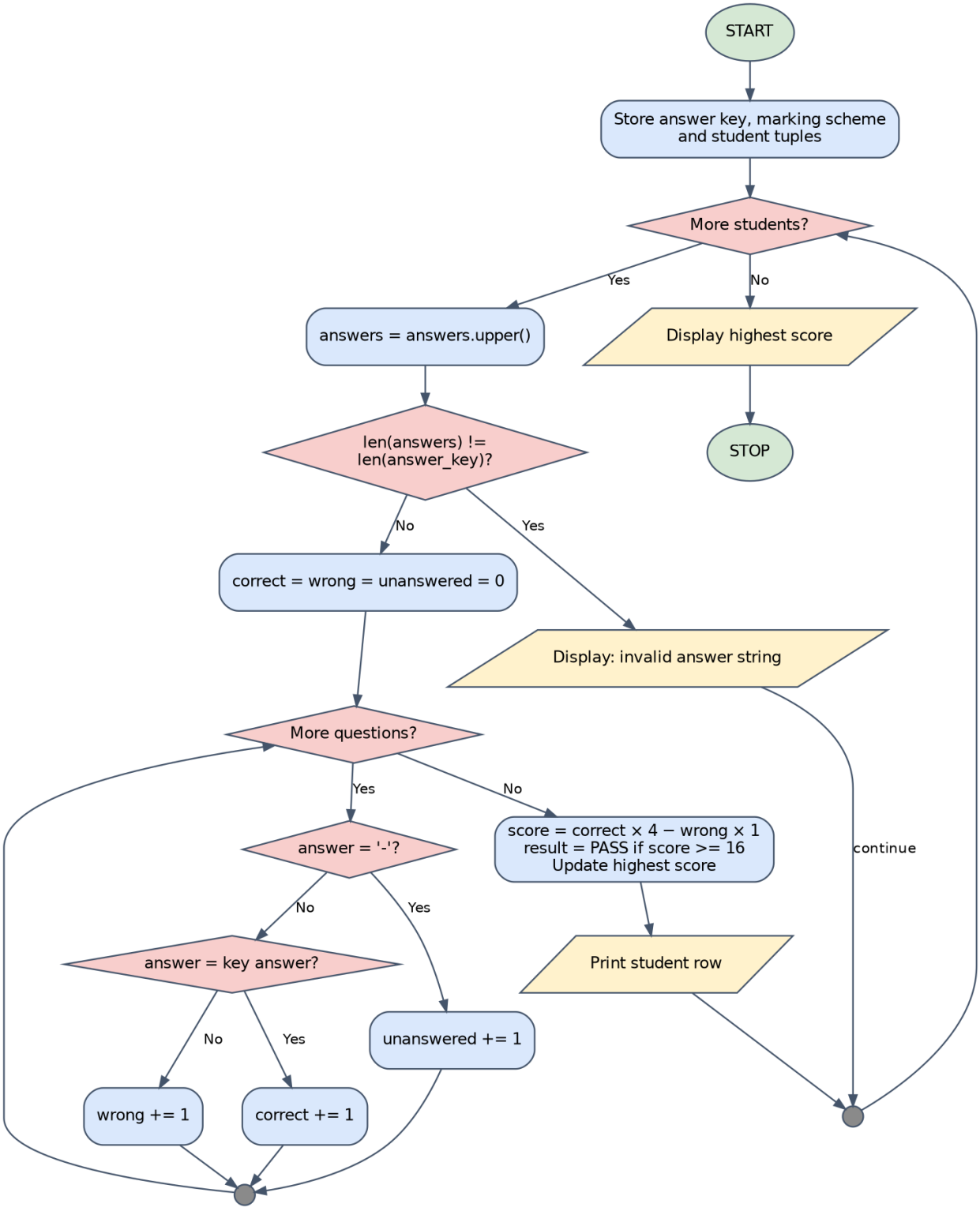
4. Algorithm

1. Store the answer key, marking scheme and student tuples (name, answers).
2. Convert the answer string of the student to upper case.
3. If its length is different from the key, display an error and continue with the next student.
4. Set correct, wrong and unanswered to 0.
5. For every question, check the answer: a dash is unanswered, an answer equal to the key is correct, otherwise it is wrong.
6. score = correct × 4 − wrong × 1; result is PASS when the score is at least 16.
7. Display the student row and keep track of the highest score.

5. Pseudocode

```
START
  SET answer_key = "ABCDABCDAB"
  SET correct_marks = 4, negative_marks = 1, pass_score = 16
  STORE students as (name, answers)
  top_score = very small number
  FOR each (name, answers)
    answers = answers in upper case
    IF length(answers) is not equal to length(answer_key) THEN
      PRINT "Invalid answer string"
      CONTINUE
    END IF
    correct = 0, wrong = 0, unanswered = 0
    FOR i = 0 to length(answer_key) - 1
      IF answers[i] = "-" THEN unanswered = unanswered + 1
      ELSE IF answers[i] = answer_key[i] THEN correct = correct + 1
      ELSE wrong = wrong + 1
    END FOR
    score = correct * correct_marks - wrong * negative_marks
    IF score >= pass_score THEN result = "PASS" ELSE result = "FAIL"
    update highest score
    PRINT student row
  END FOR
  PRINT highest score
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 31: String-Based Examination Answer Evaluation
answer_key = "ABCDABCDAB"
correct_marks = 4
negative_marks = 1
pass_score = 16

# (Student, answer string) - a dash (-) means the question was not answered
students = [
    ("Anu", "ABCDABCDAB"),
    ("Bala", "abccabdda-"),
    ("Chandu", "BBBBBBBBBB"),
    ("Divya", "A-C-A-C-A-"),
```

```
    ("Esha", "ABCDAB")
]

print(f"Answer key: {answer_key} | Maximum score: {len(answer_key) * correct_marks}")
print("-" * 66)
print(f"{'Student':<9}{'Correct':>8}{'Wrong':>7}{'Unanswered':>12}{'Score':>7}   Result")

top_name = ""
top_score = -1000

for name, answers in students:
    answers = answers.upper()

    if len(answers) != len(answer_key):
        print(f"{name:<9}Invalid answer string (expected {len(answer_key)} answers, found {len(answers)})")
        continue

    correct = 0
    wrong = 0
    unanswered = 0

    for i in range(len(answer_key)):
        if answers[i] == "-":
            unanswered += 1
        elif answers[i] == answer_key[i]:
            correct += 1
        else:
            wrong += 1

    score = correct * correct_marks - wrong * negative_marks

    if score >= pass_score:
        result = "PASS"
    else:
        result = "FAIL"

    if score > top_score:
        top_score = score
        top_name = name

    print(f"{name:<9}{correct:>8}{wrong:>7}{unanswered:>12}{score:>7}   {result}")

print("-" * 66)
print(f"Highest score: {top_name} ({top_score})")
```

8. Test Cases

Test Case	Student / Answers	Expected Result
1	Anu - ABCDABCDAB	10 correct, 0 wrong, 0 unanswered → score 40 - PASS
2	Bala - abccabdda- (lower case)	7 correct, 2 wrong, 1 unanswered → score 26 - PASS
3	Chandu - BBBBBBBBBB	3 correct, 7 wrong → 3 × 4 − 7 × 1 = 5 - FAIL
4	Divya - A-C-A-C-A-	5 correct, 0 wrong, 5 unanswered → score 20 - PASS
5	Esha - ABCDAB (6 answers)	Invalid answer string (expected 10, found 6)
6	Highest score	Anu (40)

9. Sample Output

```
Answer key: ABCDABCDAB | Maximum score: 40
------------------------------------------------------------------
Student   Correct  Wrong  Unanswered  Score   Result
Anu            10      0           0     40   PASS
Bala            7      2           1     26   PASS
Chandu          3      7           0      5   FAIL
Divya           5      0           5     20   PASS
Esha     Invalid answer string (expected 10 answers, found 6)
------------------------------------------------------------------
Highest score: Anu (40)
```

10. Conclusion

The Answer Evaluation program uses strings, indexing, string methods and loops to compare answers with a key. Negative marking and unanswered questions are handled with simple conditional statements, and length validation protects the program from invalid answer strings.

CASE STUDY 36 – REPEATED WORD AND PHRASE DETECTION

1. Problem Statement

Design a Python program that processes a paragraph, normalizes the letter case, removes punctuation, counts word occurrences using permitted string and list operations and identifies the repeated words and repeated two-word phrases along with their positions.

2. Objectives

1. Convert the paragraph to lower case and remove punctuation.
2. Split the paragraph into words.
3. Count the occurrences of every word using lists.
4. Record the positions of every word in a nested list.
5. Identify repeated words and repeated two-word phrases.
6. Use a set to find the number of distinct words.

3. Assumptions

• The paragraph is sample text stored in the program.
• Words with fewer than 3 letters (such as 'is' and 'to') are ignored while counting repeated words.
• Phrases are formed by two consecutive words and all words are used for phrases.
• Positions are numbered from 1 for the first word of the paragraph.
• A word or phrase is repeated when it occurs more than once.

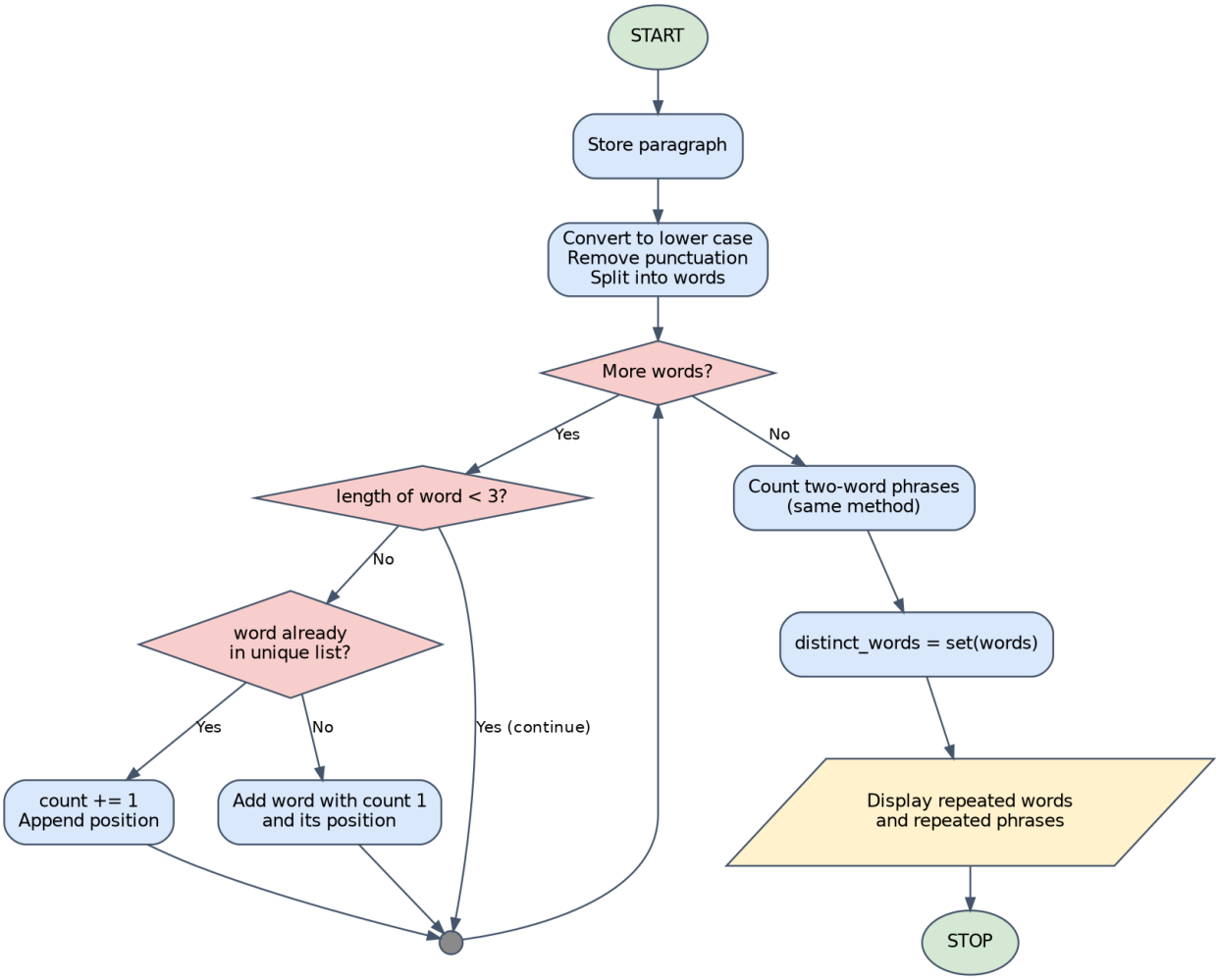
4. Algorithm

1. Store the paragraph and convert it to lower case.
2. Remove the punctuation symbols using replace().
3. Split the text into a list of words.
4. For every word of 3 or more letters, check whether it is already in the unique-word list; if yes, increase its count and add the position, otherwise add it with count 1.
5. Repeat the same method for every pair of consecutive words to count phrases.
6. Create a set of the words to find the number of distinct words.
7. Display the repeated words and phrases with counts and positions.

5. Pseudocode

```
START
  STORE paragraph
  text = paragraph in lower case
  FOR each punctuation symbol
    text = text with the symbol removed
  END FOR
  words = list of words in text
  unique_words = [], counts = [], positions = []
  FOR i = 0 to number of words - 1
    word = words[i]
    IF length(word) < 3 THEN CONTINUE
    IF word is in unique_words THEN
      increase its count and append (i + 1) to its positions
    ELSE
      append word, count 1 and positions [i + 1]
    END IF
  END FOR
  FOR i = 0 to number of words - 2
    phrase = words[i] + " " + words[i + 1]
    count the phrase in the same way
  END FOR
  distinct_words = set of words
  PRINT total words and distinct words
  PRINT every word with count > 1 and its positions
  PRINT every phrase with count > 1 and its positions
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 36: Repeated Word and Phrase Detection
paragraph = ("Python is easy to learn. Python is powerful and Python is popular. "
             "Learning Python daily makes learning easy, and learning easy things "
             "builds confidence. Practice daily, practice again, and practice more.")

# Normalize: lower case and remove punctuation
text = paragraph.lower()
for symbol in ".,!?;:":
    text = text.replace(symbol, "")
words = text.split()

# Count words of 3 or more letters; keep their positions in a nested list
unique_words = []
counts = []
positions = []

for i in range(len(words)):
    word = words[i]
    if len(word) < 3:
        continue
    if word in unique_words:
        index = unique_words.index(word)
        counts[index] += 1
        positions[index].append(i + 1)
    else:
        unique_words.append(word)
        counts.append(1)
        positions.append([i + 1])

# Count two-word phrases
phrases = []
phrase_counts = []
phrase_positions = []
```

```
for i in range(len(words) - 1):
    phrase = words[i] + " " + words[i + 1]
    if phrase in phrases:
        index = phrases.index(phrase)
        phrase_counts[index] += 1
        phrase_positions[index].append(i + 1)
    else:
        phrases.append(phrase)
        phrase_counts.append(1)
        phrase_positions.append([i + 1])

distinct_words = set(words)
print("Total words    :", len(words))
print("Distinct words :", len(distinct_words))
print()

print("REPEATED WORDS (3+ letters)")
print(f"{'Word':<12}{'Count':>6}   Positions")
repeated_words = 0
for k in range(len(unique_words)):
    if counts[k] > 1:
        repeated_words += 1
        position_text = ""
        for p in positions[k]:
            if position_text != "":
                position_text += ", "
            position_text += str(p)
        print(f"{unique_words[k]:<12}{counts[k]:>6}   {position_text}")
print("Repeated words found:", repeated_words)

print()
print("REPEATED PHRASES (two words)")
print(f"{'Phrase':<18}{'Count':>6}   Starts at word")
repeated_phrases = 0
for k in range(len(phrases)):
    if phrase_counts[k] > 1:
        repeated_phrases += 1
        position_text = ""
        for p in phrase_positions[k]:
            if position_text != "":
                position_text += ", "
            position_text += str(p)
        print(f"{phrases[k]:<18}{phrase_counts[k]:>6}   {position_text}")
print("Repeated phrases found:", repeated_phrases)
```

8. Test Cases

Test Case	Item	Expected Result
1	Total and distinct words	31 words in total, 17 distinct words (found using a set)
2	Word 'python'	Count 4 at positions 1, 6, 10, 14
3	Word 'practice'	Count 3 at positions 25, 27, 30
4	Word 'daily'	Count 2 at positions 15, 26
5	Short words such as 'is', 'to'	Ignored while counting repeated words (fewer than 3 letters)
6	Phrase 'python is'	Count 3, starts at words 1, 6, 10
7	Phrase 'learning easy'	Count 2, starts at words 17, 20

9. Sample Output

```
Total words    : 31
Distinct words : 17

REPEATED WORDS (3+ letters)
Word         Count   Positions
python           4   1, 6, 10, 14
easy             3   3, 18, 21
and              3   9, 19, 29
learning         3   13, 17, 20
daily            2   15, 26
```

```
practice         3   25, 27, 30
Repeated words found: 6

REPEATED PHRASES (two words)
Phrase             Count   Starts at word
python is              3   1, 6, 10
learning easy          2   17, 20
Repeated phrases found: 2
```

10. Conclusion

The Repeated Word and Phrase Detection program uses string methods, lists, nested lists, the membership operator and a set to analyse text. It shows how simple list operations can count and locate words without any advanced data structure.

CASE STUDY 37 – NESTED LOOP CALENDAR GENERATOR

1. Problem Statement

Design a Python program that generates a formatted monthly calendar using the month name, year, starting weekday and number of days entered by the user. The program aligns the dates under the weekday names with nested loops and validates the inputs.

2. Objectives

1. Read the month name, year, starting weekday and number of days.
2. Validate the starting weekday and the number of days.
3. Print a centred calendar title and the weekday header.
4. Use nested loops to print the weeks and the days.
5. Leave blank cells before the first day and after the last day.

3. Assumptions

• The week starts on Monday: 1 = Monday, 2 = Tuesday, ... 7 = Sunday.
• The starting weekday is the weekday of the first date of the month.
• A valid month has 28 to 31 days.
• Every date is printed in a field of width 3 so that all columns stay aligned.

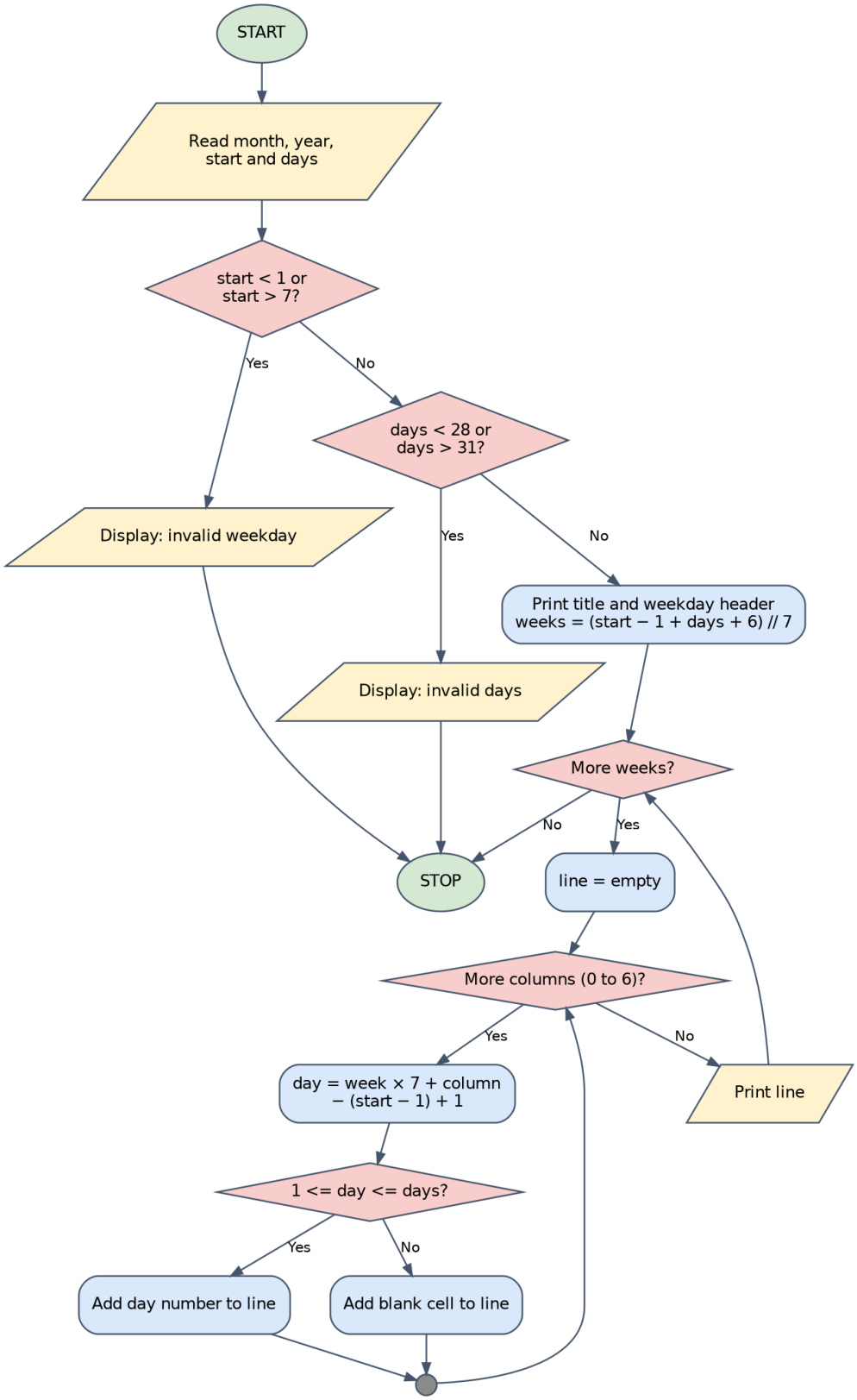
4. Algorithm

1. Read the month, year, starting weekday and number of days.
2. If the starting weekday is not from 1 to 7, display an error and stop.
3. If the number of days is not from 28 to 31, display an error and stop.
4. Print the title (month and year) centred and the weekday names.
5. Calculate the number of weeks = (start − 1 + days + 6) // 7.
6. For every week and every column from 0 to 6, calculate the date number = week × 7 + column − (start − 1) + 1.
7. If the date is from 1 to days, add it to the line; otherwise add blanks.
8. Print each completed line.

5. Pseudocode

```
START
  READ month, year, start, days
  IF start < 1 OR start > 7 THEN
    PRINT "Invalid starting weekday"
  ELSE IF days < 28 OR days > 31 THEN
    PRINT "Invalid number of days"
  ELSE
    PRINT month and year (centred)
    PRINT weekday names Mo Tu We Th Fr Sa Su
    weeks = (start - 1 + days + 6) // 7
    FOR week = 0 to weeks - 1
      line = empty text
      FOR column = 0 to 6
        day = week * 7 + column - (start - 1) + 1
        IF day >= 1 AND day <= days THEN
          line = line + day (width 3)
        ELSE
          line = line + 3 blank spaces
        END IF
      END FOR
      PRINT line
    END FOR
  END IF
STOP
```

6. Flowchart


START
Read month, year, start and days
start < 1 or start > 7?
Yes
No
days < 28 or days > 31?
Display: invalid weekday
Yes
No
Print title and weekday header weeks = (start − 1 + days + 6) // 7
Display: invalid days
More weeks?
No
Yes
STOP
line = empty
More columns (0 to 6)?
Yes
No
day = week × 7 + column − (start − 1) + 1
Print line
1 <= day <= days?
Yes
No
Add day number to line
Add blank cell to line

7. Python Source Code

```
# Case Study 37: Nested Loop Calendar Generator
month = input("Enter month name: ")
year = int(input("Enter year: "))
start = int(input("Enter starting weekday (1=Mon ... 7=Sun): "))
days = int(input("Enter number of days (28-31): "))

if start < 1 or start > 7:
    print("Invalid starting weekday. Enter a number from 1 to 7.")
elif days < 28 or days > 31:
    print("Invalid number of days. Enter a number from 28 to 31.")
else:
    day_names = ["Mo", "Tu", "We", "Th", "Fr", "Sa", "Su"]

    print()
    print((month + " " + str(year)).center(21))

    header = ""
    for name in day_names:
        header = header + f"{name:>3}"
    print(header)

    weeks = (start - 1 + days + 6) // 7

    for week in range(weeks):
        line = ""
        for column in range(7):
            day = week * 7 + column - (start - 1) + 1
            if day >= 1 and day <= days:
                line = line + f"{day:>3}"
            else:
                line = line + "   "
        print(line.rstrip())
```

8. Test Cases

Test Case	Input (month, year, start, days)	Expected Result
1	October, 2026, 4, 31	Date 1 under Th; 5 rows of weeks; last date 31 under Sa
2	February, 2026, 7, 28	Date 1 under Su; 5 rows of weeks; last date 28 under Sa
3	May, 2026, 8, 31	Invalid starting weekday. Enter a number from 1 to 7.
4	April, 2026, 3, 32	Invalid number of days. Enter a number from 28 to 31.
5	May, 2026, 3, 27	Invalid number of days. Enter a number from 28 to 31.

9. Sample Output

Sample Run 1 (input: October, 2026, 4, 31)

```
Enter month name: October
Enter year: 2026
Enter starting weekday (1=Mon ... 7=Sun): 4
Enter number of days (28-31): 31

     October 2026
Mo Tu We Th Fr Sa Su
           1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
```

Sample Run 2 (input: February, 2026, 7, 28)

```
Enter month name: February
Enter year: 2026
Enter starting weekday (1=Mon ... 7=Sun): 7
Enter number of days (28-31): 28

    February 2026
Mo Tu We Th Fr Sa Su
```

```
                   1
  2  3  4  5  6  7  8
  9 10 11 12 13 14 15
 16 17 18 19 20 21 22
 23 24 25 26 27 28
```

Sample Run 3 (input: April, 2026, 3, 32)

```
Enter month name: April
Enter year: 2026
Enter starting weekday (1=Mon ... 7=Sun): 3
Enter number of days (28-31): 32
Invalid number of days. Enter a number from 28 to 31.
```

10. Conclusion

The Calendar Generator uses nested for loops, range(), arithmetic with floor division and string formatting to print aligned weeks. The formula for the date number shows how pattern generation can be controlled by simple calculations on the row and column positions.

CASE STUDY 43 – MULTI-BRANCH SALES MATRIX COMPARISON

1. Problem Statement

Design a Python program that stores the daily sales of several products in several branches using nested lists. The program calculates product totals, compares branches, identifies the best-performing product of every branch and calculates the day-wise sales.

2. Objectives

1. Represent branch, product and day-wise sales using nested lists.
2. Calculate the total units of every product in every branch.
3. Calculate branch totals and product totals.
4. Identify the best-performing product of every branch.
5. Identify the best branch and the best product overall.
6. Calculate the total sales of every day.

3. Assumptions

- There are four branches (Hyderabad, Vijayawada, Guntur, Nellore), four products and three days of sales.
- sales[branch][product][day] stores the number of units sold.
- Sales are measured in units sold.
- If two products have the same total, the product listed first is considered the best.

4. Algorithm

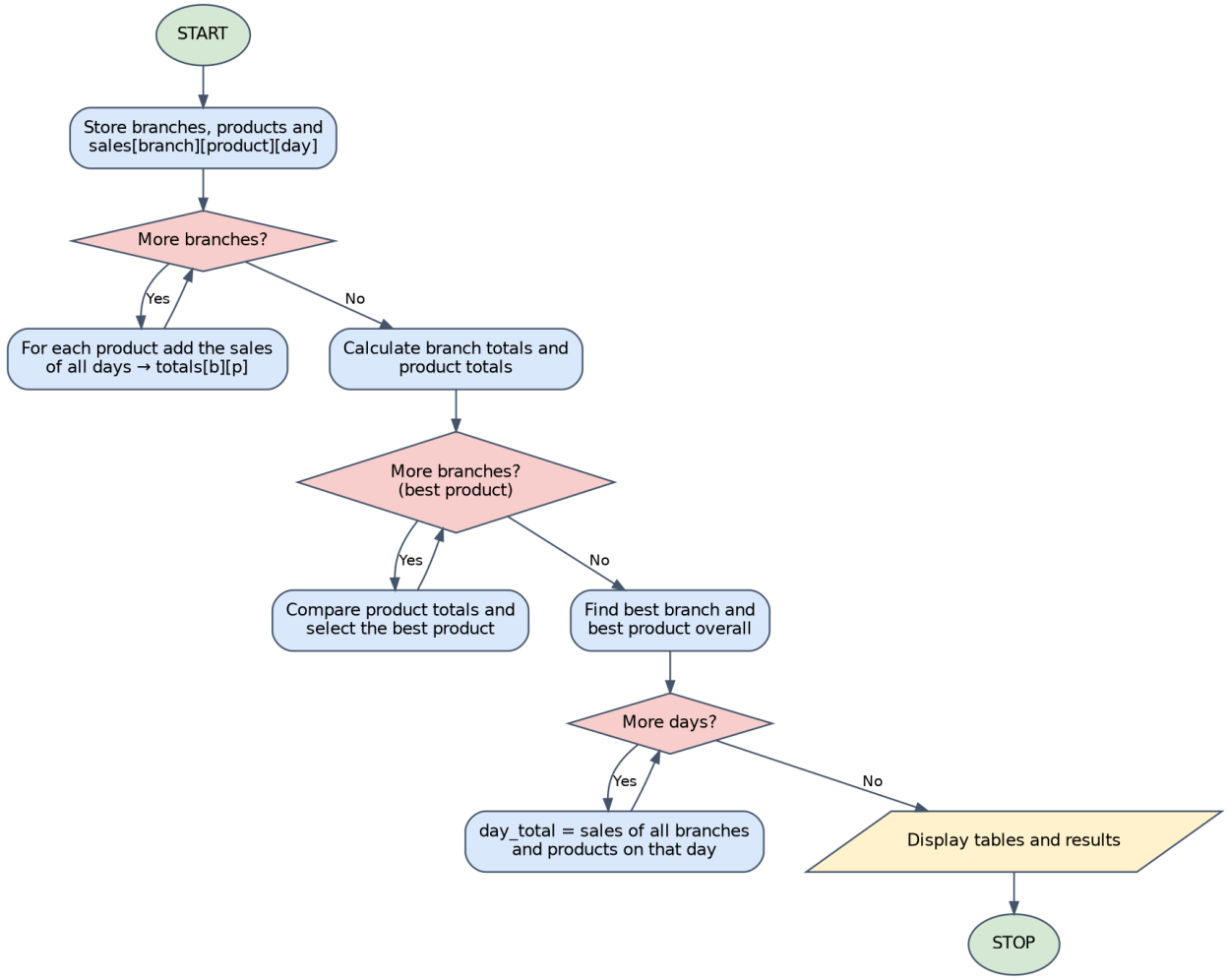
1. Store the branch names, product names and the sales nested list.
2. For every branch and every product, add the sales of the three days and store the total in a totals list.
3. Calculate the branch totals (row sums) and the product totals (column sums).
4. Print the table of totals.
5. For every branch, compare the product totals to find the best product.
6. Compare the branch totals and the product totals to find the best branch and the best product.
7. For every day, add the sales of all branches and products and display the day-wise total.

5. Pseudocode

```
START
  STORE branches, products and sales[branch][product][day]
  FOR each branch b
    row = empty list
    FOR each product p
      total = 0
      FOR each day d
        total = total + sales[b][p][d]
      END FOR
      append total to row
    END FOR
    append row to totals
  END FOR
  FOR each branch b: branch_total = sum of totals[b]
  FOR each product p: product_total = sum of totals[b][p] over all branches
  PRINT the table of totals
  FOR each branch b
    best = 0
    FOR each product p from 1
      IF totals[b][p] > totals[b][best] THEN best = p
    END FOR
    PRINT best product of the branch
  END FOR
  FIND best branch and best product overall
  FOR each day d
    day_total = sum of sales[b][p][d] for all b and p
    PRINT day_total
```

```
  END FOR
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 43: Multi-Branch Sales Matrix Comparison
branches = ["Hyderabad", "Vijayawada", "Guntur", "Nellore"]
products = ["Laptop", "Mobile", "Tablet", "Headphones"]
days = 3

# sales[branch][product] = [units sold on day 1, day 2, day 3]
sales = [
    [[5, 7, 6], [20, 25, 22], [8, 6, 9], [30, 28, 35]],
    [[3, 4, 2], [18, 15, 20], [5, 7, 6], [22, 25, 20]],
    [[2, 3, 4], [12, 14, 10], [4, 3, 5], [15, 18, 16]],
    [[4, 2, 3], [10, 12, 15], [6, 5, 7], [12, 10, 14]]
]

# Total units of every product in every branch
totals = []
for b in range(len(branches)):
    row = []
    for p in range(len(products)):
        total = 0
        for d in range(days):
            total += sales[b][p][d]
        row.append(total)
    totals.append(row)

print("PRODUCT TOTALS BY BRANCH (units sold in 3 days)")
print("-" * 70)
print(f"{'Branch':<12}{'Laptop':>8}{'Mobile':>8}{'Tablet':>8}{'Headphones':>12}{'Total':>8}")

branch_totals = []
for b in range(len(branches)):
```

```
    branch_total = 0
    for p in range(len(products)):
        branch_total += totals[b][p]
    branch_totals.append(branch_total)
    print(f"{branches[b]:<12}{totals[b][0]:>8}{totals[b][1]:>8}{totals[b][2]:>8}{totals[b][3]:>12}{branch_total:>8}")

product_totals = []
for p in range(len(products)):
    product_total = 0
    for b in range(len(branches)):
        product_total += totals[b][p]
    product_totals.append(product_total)
print("-" * 70)
print(f"{'All branches':<12}{product_totals[0]:>8}{product_totals[1]:>8}{product_totals[2]:>8}{product_totals[3]:>12}{s

print()
print("BEST-PERFORMING PRODUCT IN EACH BRANCH")
for b in range(len(branches)):
    best = 0
    for p in range(1, len(products)):
        if totals[b][p] > totals[b][best]:
            best = p
    print(f"{branches[b]:<12}{products[best]} ({totals[b][best]} units)")

best_branch = 0
for b in range(1, len(branches)):
    if branch_totals[b] > branch_totals[best_branch]:
        best_branch = b
best_product = 0
for p in range(1, len(products)):
    if product_totals[p] > product_totals[best_product]:
        best_product = p

print()
print("Best branch  :", branches[best_branch], "-", branch_totals[best_branch], "units")
print("Best product :", products[best_product], "-", product_totals[best_product], "units")

print()
print("DAY-WISE TOTAL (all branches and products)")
for d in range(days):
    day_total = 0
    for b in range(len(branches)):
        for p in range(len(products)):
            day_total += sales[b][p][d]
    print(f"Day {d + 1}: {day_total} units")
```

8. Test Cases

Test Case	Item	Expected Result
1	Branch totals	Hyderabad 201, Vijayawada 147, Guntur 106, Nellore 100 units
2	Product totals (all branches)	Laptop 45, Mobile 193, Tablet 71, Headphones 245; grand total 554
3	Best product in Hyderabad	Headphones (93 units)
4	Best product in Nellore	Mobile (37 units, ahead of Headphones with 36)
5	Best branch and best product	Hyderabad (201 units) and Headphones (245 units)
6	Day-wise totals	Day 1: 176, Day 2: 184, Day 3: 194 units (sum 554)

9. Sample Output

```
PRODUCT TOTALS BY BRANCH (units sold in 3 days)
----------------------------------------------------------------------
Branch        Laptop  Mobile  Tablet  Headphones   Total
Hyderabad         18      67      23          93     201
Vijayawada         9      53      18          67     147
Guntur             9      36      12          49     106
Nellore            9      37      18          36     100
----------------------------------------------------------------------
All branches      45     193      71         245     554

BEST-PERFORMING PRODUCT IN EACH BRANCH
```

```
Hyderabad   Headphones (93 units)
Vijayawada  Headphones (67 units)
Guntur      Headphones (49 units)
Nellore     Mobile (37 units)

Best branch  : Hyderabad - 201 units
Best product : Headphones - 245 units

DAY-WISE TOTAL (all branches and products)
Day 1: 176 units
Day 2: 184 units
Day 3: 194 units
```

10. Conclusion

The Sales Matrix program uses three-level nested lists and nested loops to analyse sales by branch, product and day. Comparison operators help to find the best product and the best branch, showing how nested lists can organise tabular data.

CASE STUDY 48 – TUPLE-BASED EMPLOYEE SHIFT CONFLICT DETECTION

1. Problem Statement

Design a Python program that represents employee shifts as tuples, validates the time intervals, compares every pair of shifts, detects overlapping assignments and identifies the employees with conflicting schedules.

2. Objectives

1. Represent every shift as a tuple (employee, day, start hour, end hour).
2. Validate the shift timings.
3. Compare shifts using nested loops.
4. Detect overlapping shifts of the same employee on the same day.
5. Calculate the number of overlapping hours.
6. List the employees with conflicting schedules.

3. Assumptions

• Time is written in the 24-hour clock using whole hours.
• A valid shift starts before it ends and lies between 0 and 24; overnight shifts are not considered.
• Two shifts overlap when start1 < end2 and start2 < end1 for the same employee on the same day.
• Shifts that only touch each other (for example 8-12 and 12-16) do not conflict.

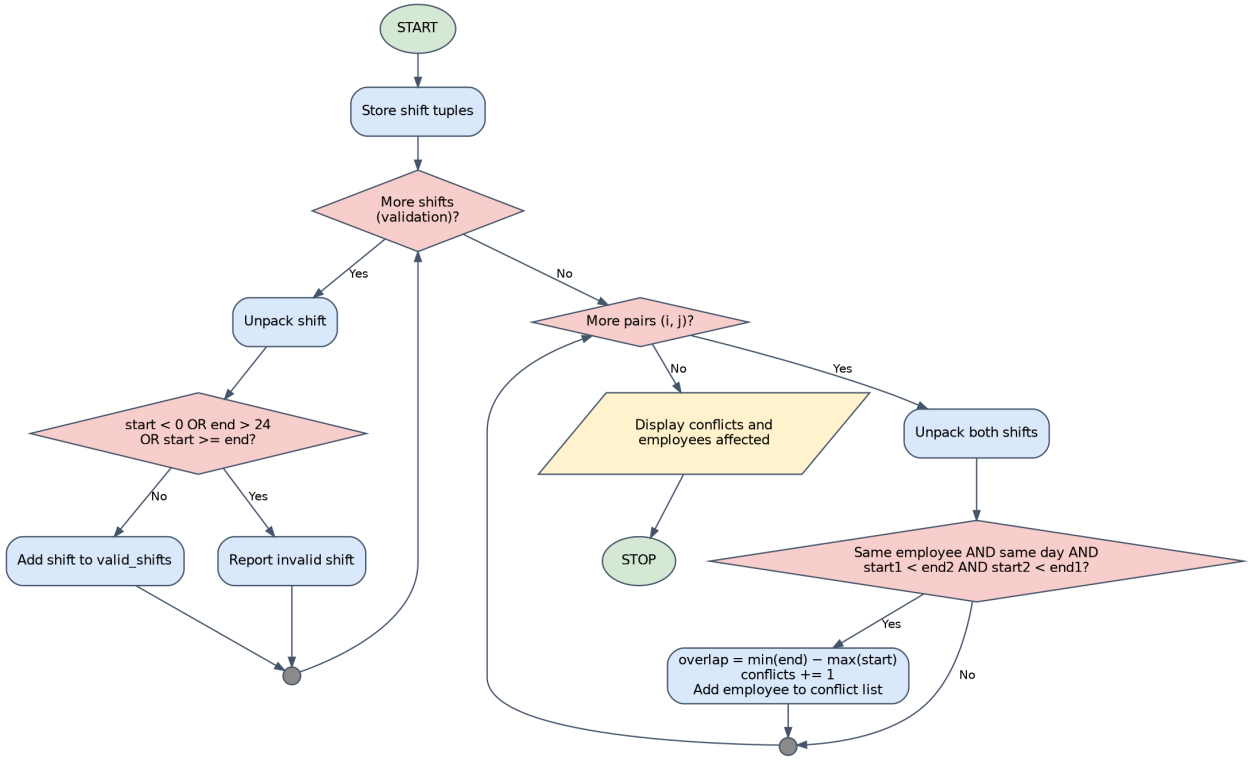
4. Algorithm

1. Store the shifts as tuples in a list.
2. For every shift, unpack the tuple and check start >= end, start < 0 and end > 24; report invalid shifts and keep the valid ones in a new list.
3. Use two nested loops to take every pair of valid shifts.
4. Unpack both shifts and check the same employee, the same day and the overlap condition.
5. For an overlap, calculate overlap start = max(start1, start2), overlap end = min(end1, end2) and the overlapping hours.
6. Add the employee to the conflict list when not already present and increase the conflict count.
7. Display all conflicts and the employees with conflicting schedules.

5. Pseudocode

```
START
  STORE shifts as (employee, day, start, end)
  valid_shifts = empty list
  FOR each shift in shifts
    UNPACK name, day, start, end
    IF start < 0 OR end > 24 OR start >= end THEN
      PRINT invalid shift
    ELSE
      append shift to valid_shifts
    END IF
  END FOR
  conflicts = 0, conflict_employees = empty list
  FOR i = 0 to number of valid shifts - 1
    FOR j = i + 1 to number of valid shifts - 1
      UNPACK shift i and shift j
      IF name1 = name2 AND day1 = day2 AND start1 < end2 AND start2 < end1 THEN
        overlap_start = larger of start1 and start2
        overlap_end = smaller of end1 and end2
        conflicts = conflicts + 1
        IF name1 is not in conflict_employees THEN append name1
        PRINT conflict and overlapping hours
      END IF
    END FOR
  END FOR
  PRINT conflicts and conflict_employees
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 48: Tuple-Based Employee Shift Conflict Detection

# (Employee, Day, Start Hour, End Hour) - 24-hour clock
shifts = [
    ("Ravi", "Mon", 9, 17),
    ("Ravi", "Mon", 15, 20),
    ("Ravi", "Tue", 9, 13),
    ("Sita", "Mon", 8, 12),
    ("Sita", "Mon", 12, 16),
    ("Sita", "Wed", 14, 10),
    ("Kiran", "Mon", 10, 18),
    ("Kiran", "Mon", 9, 11),
    ("Kiran", "Mon", 17, 19),
    ("Meena", "Thu", 6, 14)
]

# Step 1: validate every shift
valid_shifts = []
print("SHIFT VALIDATION")
print("-" * 60)
for shift in shifts:
    name, day, start, end = shift
    if start < 0 or end > 24 or start >= end:
        print(f"INVALID: {name} {day} {start}:00-{end}:00 (start must be before end, 0-24 only)")
    else:
        valid_shifts.append(shift)
print("Valid shifts:", len(valid_shifts), "| Invalid shifts:", len(shifts) - len(valid_shifts))

# Step 2: compare every pair of valid shifts
print()
print("CONFLICT DETECTION")
print("-" * 60)
conflicts = 0
conflict_employees = []

for i in range(len(valid_shifts)):
    for j in range(i + 1, len(valid_shifts)):
        name1, day1, start1, end1 = valid_shifts[i]
        name2, day2, start2, end2 = valid_shifts[j]

        if name1 == name2 and day1 == day2 and start1 < end2 and start2 < end1:
            overlap_start = max(start1, start2)
            overlap_end = min(end1, end2)
```

```
            conflicts += 1
            if name1 not in conflict_employees:
                conflict_employees.append(name1)
            print(f"CONFLICT: {name1} on {day1}: {start1}-{end1} overlaps {start2}-{end2} "
                  f"({overlap_end - overlap_start} hour(s), {overlap_start}:00-{overlap_end}:00)")

print("-" * 60)
print("Total conflicts:", conflicts)
if len(conflict_employees) > 0:
    print("Employees with conflicting schedules:", ", ".join(conflict_employees))
else:
    print("No employee has conflicting schedules")
```

8. Test Cases

Test Case	Shifts Compared	Expected Result
1	Ravi Mon 9-17 and Mon 15-20	Conflict: 2 hour(s), 15:00-17:00
2	Ravi Mon 9-17 and Tue 9-13	No conflict (different days)
3	Sita Mon 8-12 and Mon 12-16	No conflict (shifts only touch at 12)
4	Sita Wed 14-10	Invalid shift (start is not before end)
5	Kiran Mon 10-18 and Mon 9-11	Conflict: 1 hour(s), 10:00-11:00
6	Kiran Mon 10-18 and Mon 17-19	Conflict: 1 hour(s), 17:00-18:00
7	Kiran Mon 9-11 and Mon 17-19	No conflict
8	Summary	3 conflicts; employees with conflicts: Ravi, Kiran

9. Sample Output

```
SHIFT VALIDATION
------------------------------------------------------------
INVALID: Sita Wed 14:00-10:00 (start must be before end, 0-24 only)
Valid shifts: 9 | Invalid shifts: 1

CONFLICT DETECTION
------------------------------------------------------------
CONFLICT: Ravi on Mon: 9-17 overlaps 15-20 (2 hour(s), 15:00-17:00)
CONFLICT: Kiran on Mon: 10-18 overlaps 9-11 (1 hour(s), 10:00-11:00)
CONFLICT: Kiran on Mon: 10-18 overlaps 17-19 (1 hour(s), 17:00-18:00)
------------------------------------------------------------
Total conflicts: 3
Employees with conflicting schedules: Ravi, Kiran
```

10. Conclusion

The Shift Conflict program represents shifts as tuples and uses tuple unpacking, nested loops and logical operators to detect overlapping time intervals. Validating each shift first keeps the comparison step simple and reliable.

CASE STUDY 57 – SMART CANTEEN ORDER AND STOCK MANAGEMENT

1. Problem Statement

Design a Python program that processes successive customer orders in a canteen, calculates the bills, updates the available inventory, rejects unavailable quantities and summarizes the daily sales and the remaining stock.

2. Objectives

1. Maintain item names, prices, stock and sold quantities in lists.
2. Store the orders as tuples.
3. Validate the item number and the quantity of every order.
4. Reject orders that exceed the available stock.
5. Calculate bills and update stock and sales.
6. Display low-stock alerts and a daily summary.

3. Assumptions

• Menu items, prices and initial stock are sample data.
• An order is a tuple (item number, quantity); item numbers start from 0.
• Only positive quantities are accepted and an order is accepted only if the quantity does not exceed the available stock.
• An item with fewer than 5 units left after an order raises a low-stock alert.

4. Algorithm

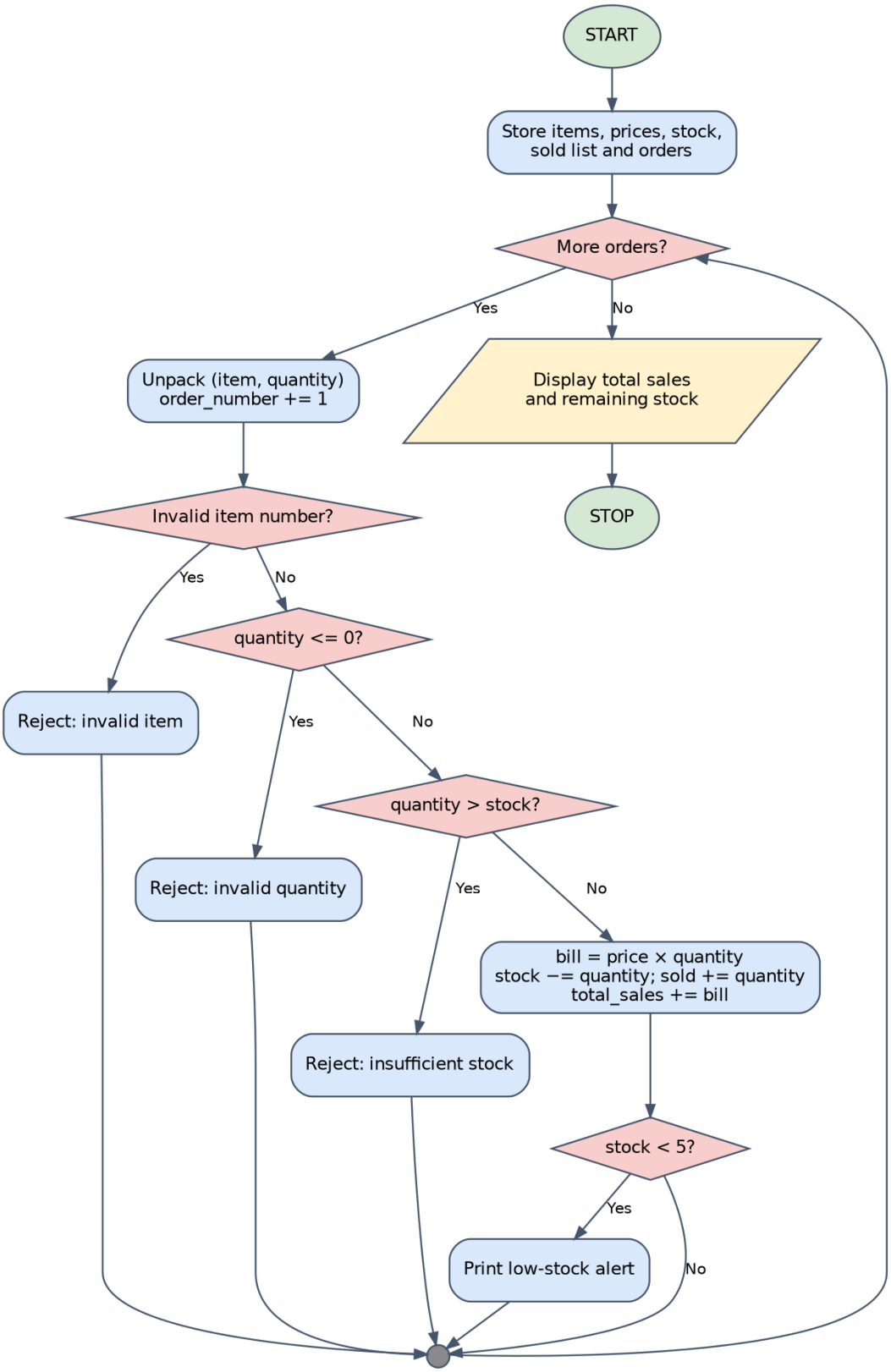
1. Store the items, prices, stock, sold lists and the orders.
2. Take each order, unpack it into item number and quantity and increase the order number.
3. If the item number is invalid, reject the order.
4. If the quantity is zero or negative, reject the order.
5. If the quantity is more than the stock, reject the order with the available quantity.
6. Otherwise calculate the bill, reduce the stock, increase the sold quantity and add the bill to the total sales.
7. Print a low-stock alert when the remaining stock is below 5.
8. After all orders display the accepted and rejected counts, total sales and the stock table.

5. Pseudocode

```
START
  STORE items, prices, stock and sold lists
  STORE orders as (item_number, quantity)
  total_sales = 0, accepted = 0, rejected = 0, order_number = 0
  FOR each (item_number, quantity) in orders
    order_number = order_number + 1
    IF item_number < 0 OR item_number >= number of items THEN
      PRINT rejected (invalid item)
      rejected = rejected + 1
    ELSE IF quantity <= 0 THEN
      PRINT rejected (invalid quantity)
      rejected = rejected + 1
    ELSE IF quantity > stock[item_number] THEN
      PRINT rejected (insufficient stock)
      rejected = rejected + 1
    ELSE
      bill = prices[item_number] * quantity
      stock[item_number] = stock[item_number] - quantity
      sold[item_number] = sold[item_number] + quantity
      total_sales = total_sales + bill
      accepted = accepted + 1
      PRINT accepted order
      IF stock[item_number] < low_stock_level THEN PRINT low stock alert
    END IF
```

```
  END FOR
  PRINT accepted, rejected, total_sales and stock table
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 57: Smart Canteen Order and Stock Management
items = ["Idli", "Dosa", "Vada", "Juice", "Coffee"]
prices = [30, 50, 20, 25, 15]
stock = [20, 10, 8, 15, 25]
sold = [0, 0, 0, 0, 0]
low_stock_level = 5

# (Item number, Quantity)
orders = [(0, 3), (1, 2), (2, 10), (3, 4), (1, 0), (7, 2), (2, 6), (4, 5), (1, 8)]

total_sales = 0
accepted = 0
rejected = 0
order_number = 0

print("CANTEEN ORDER LOG")
print("-" * 64)

for item_number, quantity in orders:
    order_number += 1

    if item_number < 0 or item_number >= len(items):
        print(f"Order {order_number}: REJECTED - invalid item number {item_number}")
        rejected += 1
    elif quantity <= 0:
        print(f"Order {order_number}: REJECTED - quantity must be positive ({items[item_number]})")
        rejected += 1
    elif quantity > stock[item_number]:
        print(f"Order {order_number}: REJECTED - only {stock[item_number]} {items[item_number]} available, requested {
        rejected += 1
    else:
        bill = prices[item_number] * quantity
        stock[item_number] -= quantity
        sold[item_number] += quantity
        total_sales += bill
        accepted += 1
        print(f"Order {order_number}: ACCEPTED - {quantity} x {items[item_number]} = Rs. {bill}")
        if stock[item_number] < low_stock_level:
            print(f"          >> Low stock alert: {items[item_number]} has {stock[item_number]} left")

print("-" * 64)
print("Accepted orders:", accepted, "| Rejected orders:", rejected)
print("Total sales    : Rs.", total_sales)
print()
print(f"{'Item':<10}{'Price':>7}{'Sold':>7}{'Remaining':>11}")
for i in range(len(items)):
    print(f"{items[i]:<10}{prices[i]:>7}{sold[i]:>7}{stock[i]:>11}")
```

8. Test Cases

Test Case	Order (item, quantity)	Expected Result
1	Order 1: Idli, 3	Accepted; bill Rs. 90; stock 17
2	Order 3: Vada, 10 (stock 8)	Rejected: only 8 Vada available
3	Order 5: Dosa, 0	Rejected: quantity must be positive
4	Order 6: item number 7	Rejected: invalid item number
5	Order 7: Vada, 6	Accepted; bill Rs. 120; low-stock alert (2 left)
6	Order 9: Dosa, 8 (stock 8)	Accepted; bill Rs. 400; stock becomes 0 - low-stock alert
7	Daily summary	6 accepted, 3 rejected; total sales Rs. 885

9. Sample Output

```
CANTEEN ORDER LOG
----------------------------------------------------------------
Order 1: ACCEPTED - 3 x Idli = Rs. 90
Order 2: ACCEPTED - 2 x Dosa = Rs. 100
Order 3: REJECTED - only 8 Vada available, requested 10
Order 4: ACCEPTED - 4 x Juice = Rs. 100
Order 5: REJECTED - quantity must be positive (Dosa)
Order 6: REJECTED - invalid item number 7
Order 7: ACCEPTED - 6 x Vada = Rs. 120
          >> Low stock alert: Vada has 2 left
Order 8: ACCEPTED - 5 x Coffee = Rs. 75
Order 9: ACCEPTED - 8 x Dosa = Rs. 400
          >> Low stock alert: Dosa has 0 left
----------------------------------------------------------------
Accepted orders: 6 | Rejected orders: 3
Total sales    : Rs. 885

Item        Price   Sold  Remaining
Idli           30      3         17
Dosa           50     10          0
Vada           20      6          2
Juice          25      4         11
Coffee         15      5         20
```

10. Conclusion

The Canteen Order program uses lists, tuples, tuple unpacking, loops and a chain of conditions to validate every order before changing the stock. It shows how parallel lists can store menu data and how the program keeps the sales and inventory consistent.